qdrg_FGHP_2026

October 12, 2026

[1]:
```
import itertools

def shortest_paths(G, D, u, v):
    """All shortest paths from u to v, each as a list of vertices."""
    if u == v:
        return [[v]]
    return [[u] + P
            for w in G.neighbors(u) if D[w][v] == D[u][v] - 1
            for P in shortest_paths(G, D, w, v)]

def path_edges(P):
    """The edges of a path, each edge as a frozenset {a, b}."""
    return [frozenset((P[i], P[i+1])) for i in range(len(P) - 1)]

def two_geodesic_cycles(G, j):
    """All two-geodesic 2j-cycles of G, each as a frozenset of edges."""
    D = G.distance_all_pairs()
    cycles = set()
    for u, v in itertools.combinations(list(G), 2):
        if D[u][v] == j:
            paths = shortest_paths(G, D, u, v)
            if len(paths) == 2:                          # exactly two shortest␣
↪paths
                C = frozenset(path_edges(paths[0]) + path_edges(paths[1]))
                if len(C) == 2*j:                        # they form a cycle of␣
↪length 2j
                    cycles.add(C)
    return list(cycles)

def parity(x):
    return bin(x).count("1") % 2

def smallest_odd_family(G, j):
    G = G.copy(); G.relabel()
    E = [frozenset(e) for e in G.edge_iterator(labels=False)]
    cycles = two_geodesic_cycles(G, j)
    p = MixedIntegerLinearProgram(maximization=False)
```

```
    z = p.new_variable(binary=True)                    # z[i] = 1 if cycle i is␣
↪in the family
    h = p.new_variable(integer=True, nonnegative=True)
    total = p.sum(z[i] for i in range(len(cycles)))
    p.set_objective(total)
    for k, e in enumerate(E):
        on_e = [z[i] for i, C in enumerate(cycles) if e in C]
        if on_e:
            p.add_constraint(p.sum(on_e) == 2*h[k])    # edge e covered an even␣
↪number of times
    p.add_constraint(total == 2*h['s'] + 1)            # odd number of cycles
    return int(round(p.solve()))
```

[2]:
```
some_graphs_Table1 = [("Icosahedron",graphs.IcosahedralGraph()),
        ("Hamming(2,3)",graphs.HammingGraph(2,3)),
        ("Shrikhande",graphs.ShrikhandeGraph()),
        ("Pappus",graphs.PappusGraph()),
        ("Dodecahedron",graphs.DodecahedralGraph()),
        ("Klein 7-regular",graphs.Klein7RegularGraph()),
        ("Coxeter",graphs.CoxeterGraph()),
        ("Kneser(7,3)",graphs.KneserGraph(7,3)),
        ("Line of Tutte's 8-cage",graphs.TutteCoxeterGraph().line_graph())
        ]
for name, G in some_graphs_Table1:
    j = list(G.is_distance_regular(True)[1]).index(2)
    s = smallest_odd_family(G,j)
    d = G.diameter()
    print("For", name,":","s = ",s,"and","j = ",j,"and", "diam =",d,".","So q␣
↪is in between",j+1,"to",d+1)
    print("")
```

```
For Icosahedron : s =  5 and j =  2 and diam = 3 . So q is in between 3 to 4

For Hamming(2,3) : s =  9 and j =  2 and diam = 2 . So q is in between 3 to 3

For Shrikhande : s =  9 and j =  2 and diam = 2 . So q is in between 3 to 3

For Pappus : s =  5 and j =  3 and diam = 4 . So q is in between 4 to 5

For Dodecahedron : s =  3 and j =  4 and diam = 5 . So q is in between 5 to 6

For Klein 7-regular : s =  7 and j =  2 and diam = 3 . So q is in between 3 to 4

For Coxeter : s =  9 and j =  4 and diam = 4 . So q is in between 5 to 5

For Kneser(7,3) : s =  9 and j =  3 and diam = 3 . So q is in between 4 to 4
```

```
For Line of Tutte's 8-cage : s =  9 and j =  4 and diam = 4 . So q is in between
5 to 5
```

[3]:
```
G = graphs.HeawoodGraph()
D = G.distance_all_pairs()
E = [frozenset(e) for e in G.edge_iterator(labels=False)]
pos = {e: i for i, e in enumerate(E)}

# Phi_3: one polynomial for each pair at distance 3; one monomial (3 edges) for␣
 ↪each shortest path
Phi3 = [[path_edges(P) for P in shortest_paths(G, D, u, v)]
        for u, v in itertools.combinations(list(G), 2) if D[u][v] == 3]

print("Computations related to Heawood Graph.")
print("variables:", len(E))                                    # expect 21
print("polynomials:", len(Phi3))                               # expect 28
print("monomials per polynomial:", set(len(f) for f in Phi3))  # expect {3}
print("occurrences of each variable:",
      set(sum(1 for f in Phi3 for m in f if e in m) for e in E))   # expect {12}

# A sign assignment is an integer a: bit i of a is 1 if edge i gets the sign -1.
# A monomial is a bit mask; its sign is the parity of the number of -1 edges on␣
 ↪it.
masks = [[int(sum(1 << pos[e] for e in m)) for m in f] for f in Phi3]

good_assignments = 0
for a in range(2**21):
    good = True
    for m1, m2, m3 in masks:
        s1 = parity(a & m1)
        if s1 == parity(a & m2) and s1 == parity(a & m3):   # all three␣
 ↪monomials have the same sign
            good = False
            break
    if good:
        good_assignments += 1

print("sign assignments with no same-sign polynomial:", good_assignments)   #␣
 ↪expect 0
```

```
Computations related to Heawood Graph.
variables: 21
polynomials: 28
monomials per polynomial: {3}
occurrences of each variable: {12}
sign assignments with no same-sign polynomial: 0
```

[4]:
```
print("Explicit matrix having three distinct eigenvalues corresponding to␣
 ↪H(3,3).")
Y = matrix(ZZ, [[-21, 12, 3], [12, 21, 3], [3, 3, 0]])
U = diagonal_matrix(ZZ, [4, -12, 8])
V = diagonal_matrix(ZZ, [10, -2, -8])
I3 = identity_matrix(ZZ, 3)
J3 = ones_matrix(ZZ, 3, 3)

N = (Y.tensor_product(I3).tensor_product(I3)
     + U.tensor_product(J3).tensor_product(I3)
     + V.tensor_product(I3).tensor_product(J3))

# Row/column number 9*x1 + 3*x2 + x3 belongs to the vertex (x1, x2, x3).
verts = list(itertools.product(range(3), repeat=3))
ham = lambda x, y: sum(1 for s, t in zip(x, y) if s != t)
print("N is symmetric:", N.is_symmetric())
print("pattern is H(3,3):",
      all((N[p, q] != 0) == (ham(verts[p], verts[q]) == 1)
          for p in range(27) for q in range(27) if p != q))

print("N^3 = 603 N + 216 I:", N^3 == 603*N + 216*identity_matrix(ZZ, 27))
mp = N.change_ring(QQ).minpoly()
print("minimal polynomial:", mp)                        # expect x^3 - 603*x -␣
 ↪216
print("characteristic polynomial:", N.charpoly().factor())   # expect (x^3 -␣
 ↪603*x - 216)^9
print("eigenvalues:", mp.roots(RR, multiplicities=False))    # three distinct␣
 ↪real numbers
```

```
Explicit matrix having three distinct eigenvalues corresponding to H(3,3).
N is symmetric: True
pattern is H(3,3): True
N^3 = 603 N + 216 I: True
minimal polynomial: x^3 - 603*x - 216
characteristic polynomial: (x^3 - 603*x - 216)^9
eigenvalues: [-24.3749552878217, -0.358285227972259, 24.7332405157939]
```

[ ]: